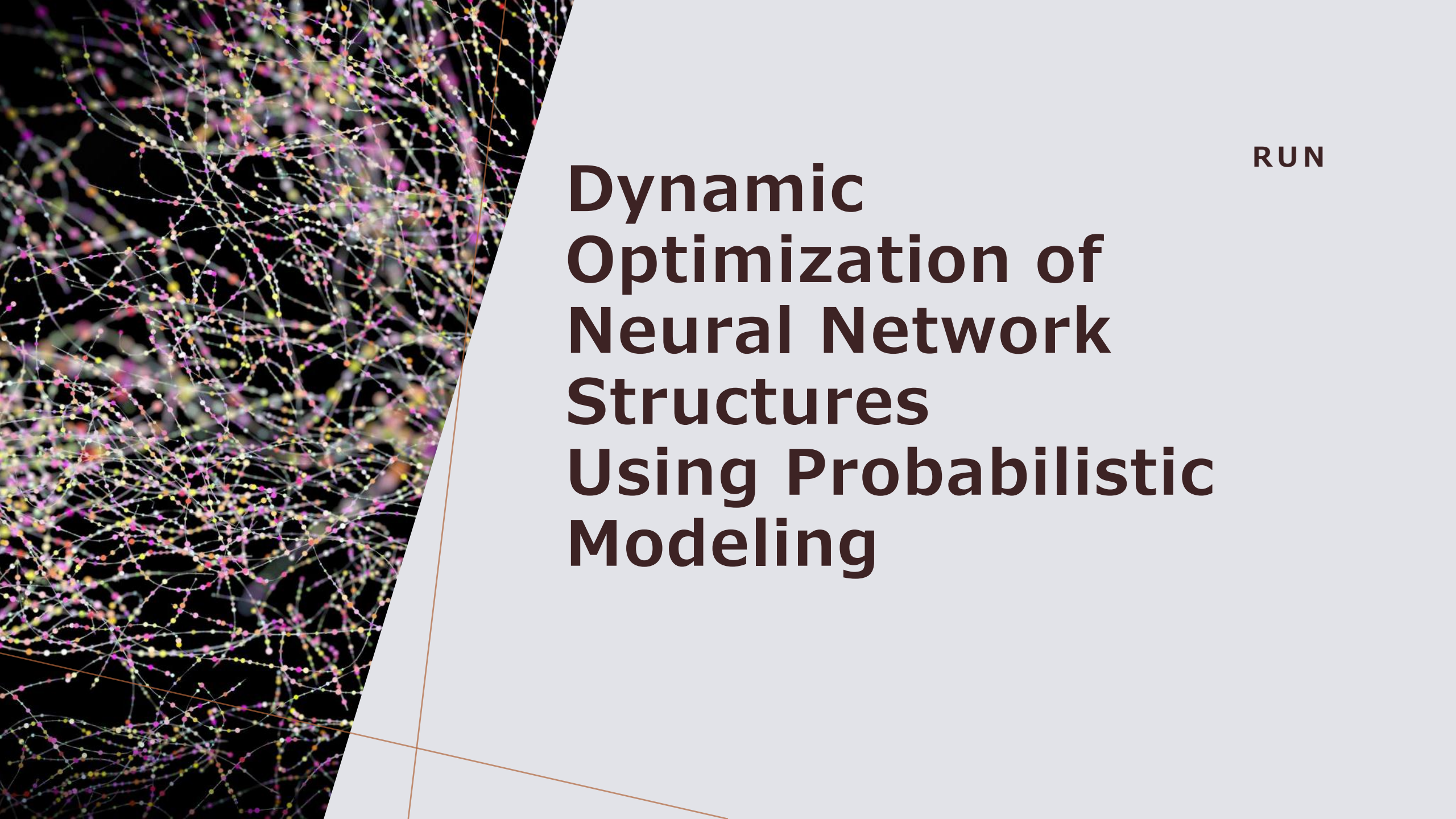




RUN

Dynamic Optimization of Neural Network Structures Using Probabilistic Modeling

DNN

DNN(ディープニューラルネットワーク)は,画像認識や自然言語処理などの様々なタスクで成功を収めている深層学習の1種

NNは入力層,隠れ層,出力層の3層で構成されているが,DNNはNNを多層化したモデル

背景

ハイパーパラメータを含む適切なネットワーク構造をユーザーが選択するのは容易ではない

静的最適化

進化的アルゴリズムなどによって接続と層の種類を最適化すること

訓練中にネットワークの構造が変わらない

合理的な構成が見つかるまで異なるネットワーク構成で学習を繰り返すため、効率が悪い

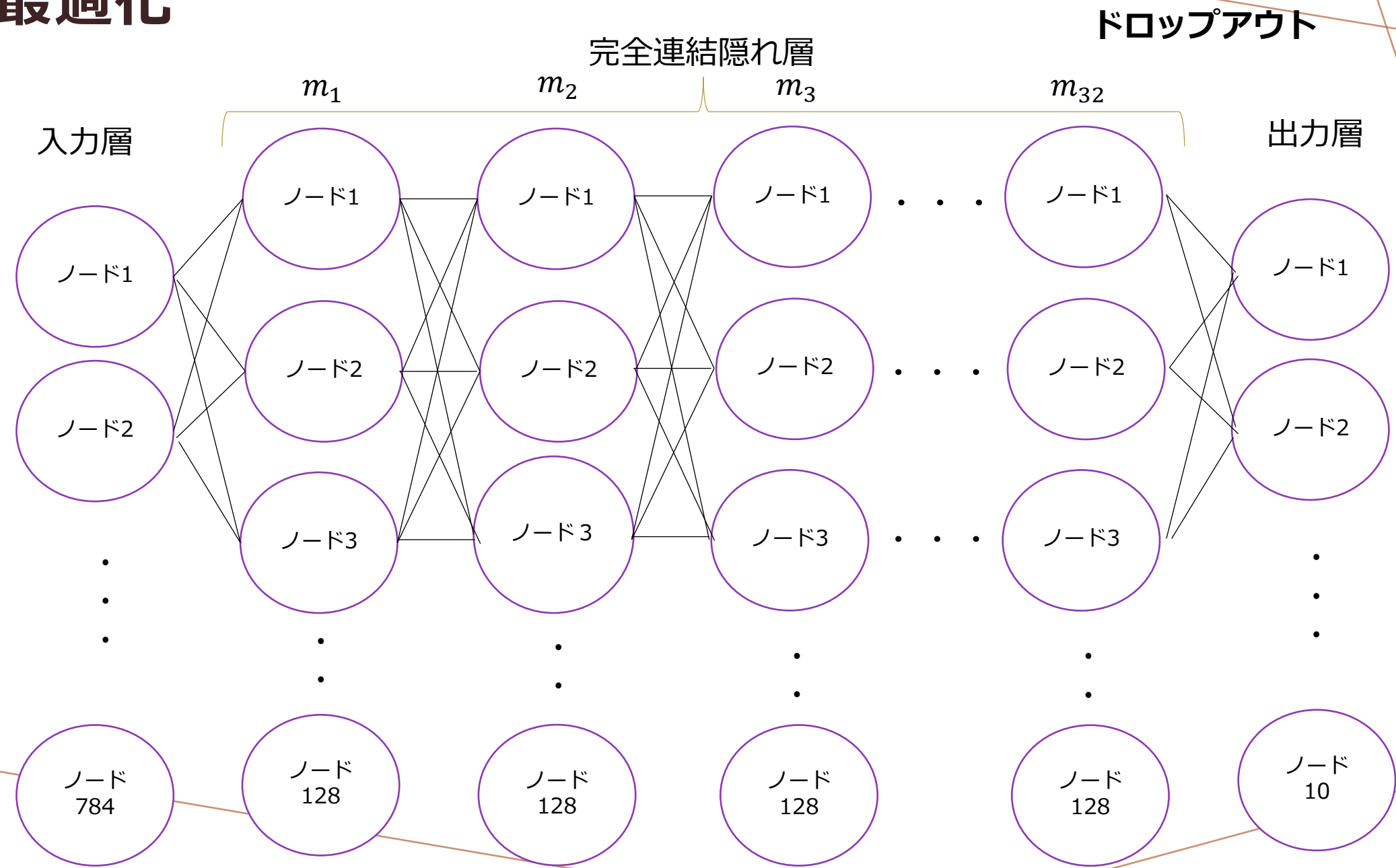
動的最適化

接続重みとニューラルネットワーク構造を同時に最適化

訓練中にネットワークの構造が逐次的に変化する

静的最適化に比べ1回の学習ループ内で接続重みとネットワーク構造を最適化するため、計算効率が高くなる

動的最適化



目的

動的最適化の計算効率をもっと早くする

ニューラルネットワーク学習中にネットワーク構造と重みパラメータを同時に最適化するためのフレームワークを新たに提案

提案手法

- ① ネットワーク構造を生成するパラメトリック分布を導入し, 分布パラメータをハイパーパラメータとして扱う.
- ② 重みとハイパーパラメータの目的関数は, 分布のもとでの損失関数の観察によって定義される.
- ③ 勾配ベースの検索アルゴリズムを適用する.
- ④ 柔軟性と効率性を示すためにベルヌーイ分布を考慮し, 提案手法が同じフレームワークのもとで最適化できることを目指す

一般的なフレームワーク

ニューラルネットワーク $\phi(W, M)$ を, 接続重みからなるベクトル $W \in W$ と各ユニットの接続性, 各ユニットの活性化関数の種類など, ネットワークの構造を決定する d 個のハイパーパラメータからなるベクトル $M \in M$ の二つのパラメータベクトルでモデル化したものを考える.

- ・ 損失 $L(W, M)$ を最小化する式

$$D = \{z_1, z_2, \dots, z_i\}$$

$$L(W, M) = \int_D l(z, W, M) p(z) dz$$

式の目的

モデルのパラメータである W, M を調整して全体の損失を最小化すること

損失関数 $L(W, M)$ は、確率密度関数 $p(z)$ のもとでデータセット D 上の関数 $l(z, W, M)$ を積分する形をしている

損失関数 $l(z, W, M)$: D 上にある各点 z での損失を表す関数のこと。

モデルである重みパラメータ W と構造パラメータ M に依存する。

確率密度関数 $p(z)$: 変数 z の確率分布を表す。これが損失関数 $L(W, M)$ の重みにあたる

実数ベクトル $\theta \in \Theta$ によってパラメータ化された M の確率分布群 $p_\theta(M)$ を考える.

$L(W, M)$ を直接最適化する代わりに, $p_\theta(M)$ のもとで期待される損失 $G(W, \theta)$ を最小限に抑えることを考慮する.

$$G(W, \theta) = \int_M L(W, M) p_\theta(M) dM$$

- ・期待値 $G(W, \theta)$ を表す式

$$G(W, \theta) = \int_M L(W, M) p_\theta(M) dM$$

式の目的

生成モデルのパラメータ θ を調整して、期待される損失 $G(W, \theta)$ を最小化すること



生成モデルを通じて損失が最小化されるようにモデルのパラメータ θ を調整して、期待される損失を最小にすることを意味している

生成モデル $p_\theta(M)$ のもとで $L(W, M)$ を積分してきた位置を計算している

この期待値は、生成モデルに従って様々な M の値を考慮し、それぞれの場合における損失の期待値を計算する

生成モデル $p_\theta(M)$: モデルのパラメータ M の確率分布を表す. この生成モデルより、異なる M の値が定義される.

勾配ベースの検索アルゴリズムを適用する

- ・パラメータ W と θ の勾配ステップ式

$$\nabla_W G(W, \theta) = \int \nabla_W L(W, M) p_\theta(M) dM$$

式の目的

パラメータ W と θ に関する損失関数 $L(W, M)$ の勾配を計算し, それをもとに更新する.
損失値が最小化する方向にパラメータが調整される

- ・パラメータ W と θ の自然勾配ステップ式

$$\tilde{\nabla}_\theta G(W, \theta) = \int L(W, M) \tilde{\nabla}_\theta \ln p_\theta(M) p_\theta(M) dM$$

式の目的

損失関数 $G(W, \theta)$ に対する θ の自然勾配を計算し, これを用いて θ を更新
自然勾配は, 通常の勾配情報を各パラメータのスケーリングに応じて調整する
各パラメータが異なる尺度で変化する場合でも適切な方向に進むことが期待される

- ・ 勾配ステップ：モデルの性能を向上させるための最適なパラメータを見つけることが目指される
- ・ 自然勾配ステップ：各パラメータが異なる尺度で変化する場合でも適切な方向に進むことが期待される

先ほど求めた自然勾配ステップのコストと勾配ステップのコストの勾配は、それぞれ $\bar{L}(W, M; Z)$ と $\nabla_W \bar{L}(W, M; Z)$ として置き換えられる。各構造パラメータ M_i のコストとその勾配を推定する必要があるため異なる2つの方法を検討する

ミニバッチ学習とは

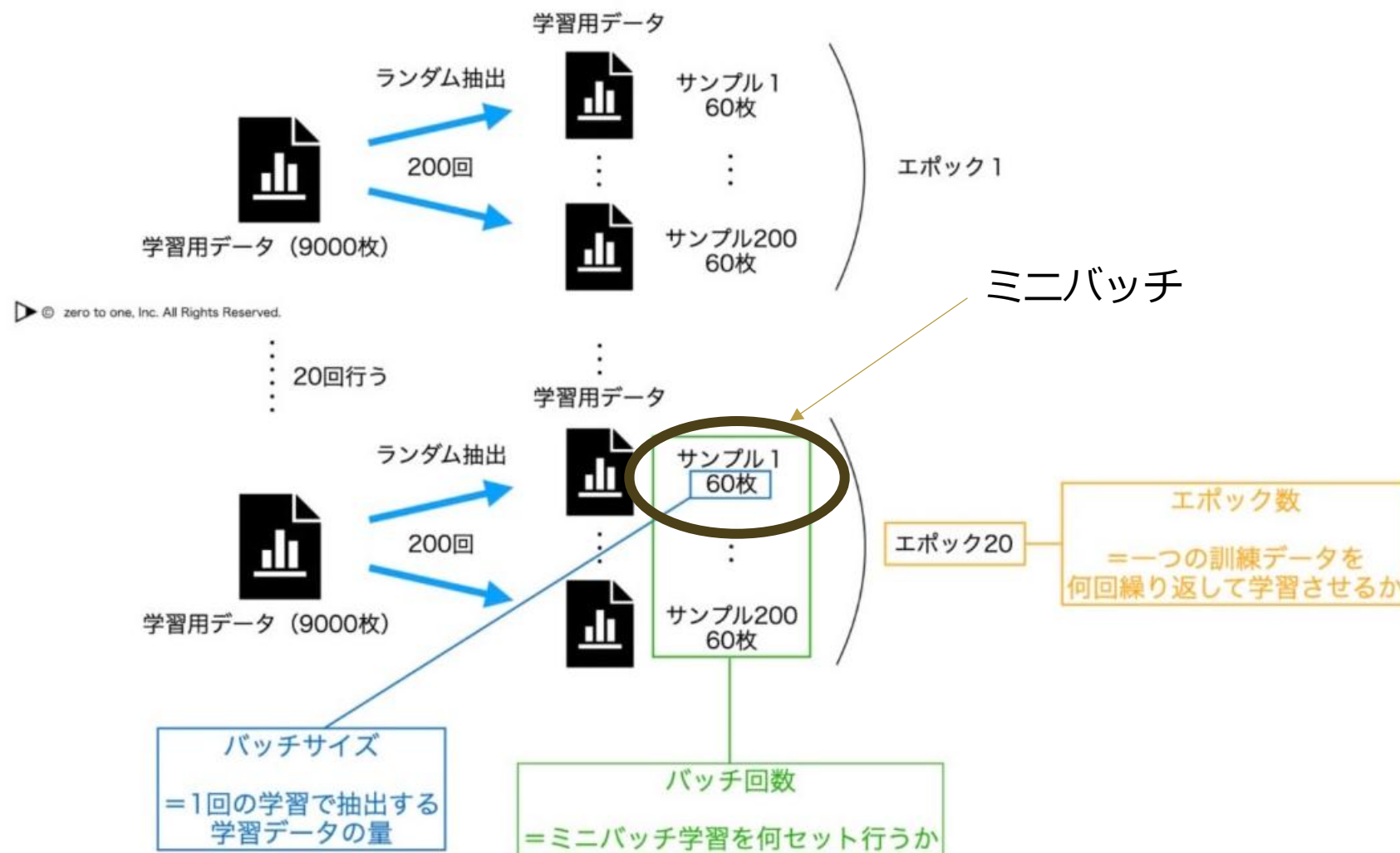
ニューラルネットワークを実施する際に頻繁に用いられる手法

訓練で用いられるデータをいくつかのグループにデータを無作為に小分けして,グループごとの損失関数を計算して平均を算出し,学習を進めること

パラメータの更新をサンプル1つ単位で行うのではなく,少数のサンプルをひとまとめにし,その単位でパラメータを更新する手法

ミニバッチ学習のイメージ

例：9000枚のデータで学習を行う場合



ミニバッチ学習 - 【AI・機械学習用語集】
(zero2one.jp)より参照

$$Z = \{z_1, \dots, z_N\}$$

- 同じミニバッチの場合

$$\bar{L}(W, M_i; Z) = \frac{1}{N} \sum_{z \in Z} l(z, W, M_i)$$

- 異なるミニバッチの場合

$$\bar{L}(W, M_i; Z_i) = \frac{\lambda}{N} \sum_{z \in Z_i} l(z, W, M_i)$$

Z : 学習データセット $\rightarrow$ ミニバッチ

z : ミニバッチ Z 中の各データ点

N : ミニバッチ内のデータ点の総数

z_1

z_2

z_3

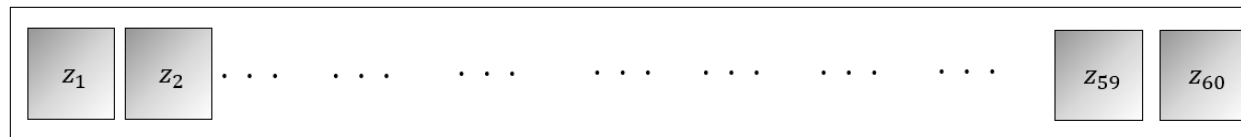
z_{9000}

$$\bar{L}(W, M_i; Z) = \frac{1}{N} \sum_{z \in Z} l(z, W, M_i)$$

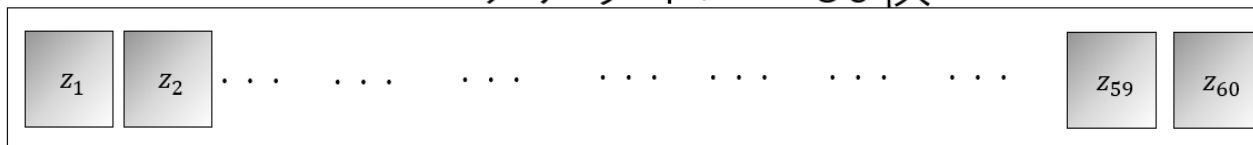
バッチサイズ : 60枚

ランダムに200回抽出
することにする

1回目



2回目

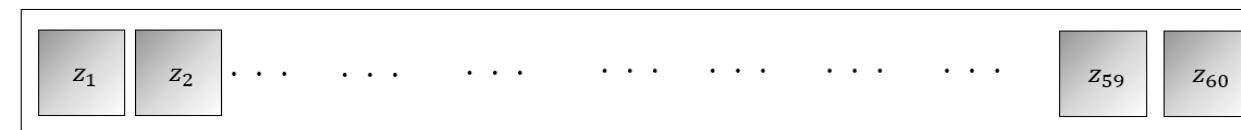


•

•

•

200回目



9000枚

z_1

z_2

z_3

z_{9000}

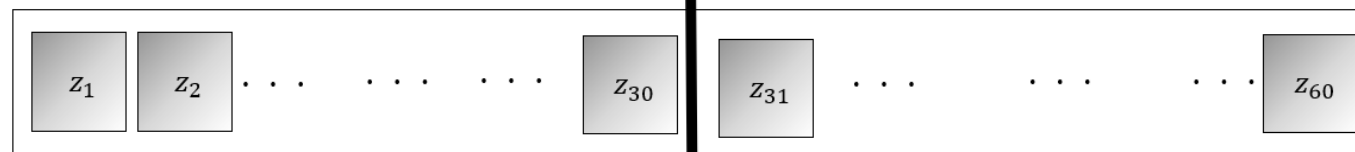
$$\bar{L}(W, M_i; Z_i) = \frac{\lambda}{N} \sum_{z \in Z_i} l(z, W, M_i)$$

$\lambda=2$

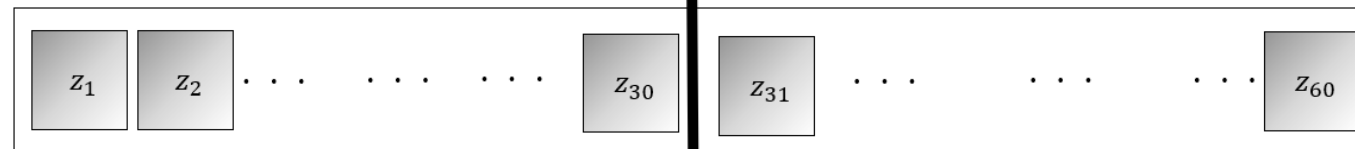
バッチサイズ : 60枚

ランダムに200回抽出
することにする

1回目

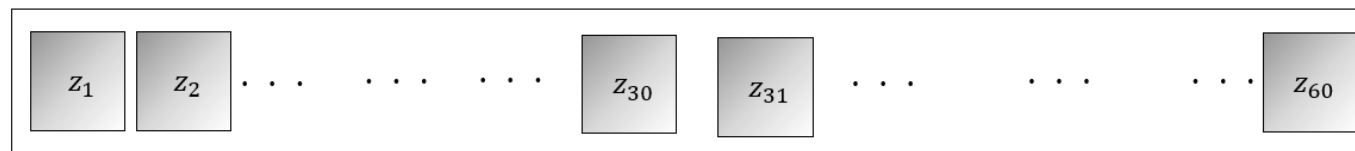


2回目



•
•
•

200回目



9000枚

- 同じミニバッチの場合

$$\bar{L}(W, M_i; Z) = \frac{1}{N} \sum_{z \in Z} l(z, W, M_i)$$

損失関数の平均を計算し、ミニバッチ内の全データに対する平均損失を得ている

全データセットではなく一部のデータのみを用いて損失を計算するため、計算効率が向上し、確率的な性質を持った勾配降下法が実現できる

- 異なるミニバッチの場合

$$\bar{L}(W, M_i; Z_i) = \frac{\lambda}{N} \sum_{z \in Z_i} l(z, W, M_i)$$

損失関数の平均を計算し、各サブセット Z_i におけるデータに対する平均損失を得ている

トレーニングデータセット Z を同じ数のデータを持つ λ 個のサブセット Z_i に分解される。各サブセット Z_i は各 M_i に使用される。

異なるデータの側面に対応する能力を獲得できる

$\bar{L}(W, M_i)$ を同じミニバッチの場合の式または異なるミニバッチの式とすると,勾配のモンテカルロ近似は以下のようになる.

$$\nabla_W G(w, \theta) \approx \frac{1}{\lambda} \sum_{i=1}^{\lambda} \nabla_W \bar{L}(W, M_i),$$
$$\tilde{\nabla}_{\theta} G(W, \theta) \approx \frac{1}{\lambda} \sum_{i=1}^{\lambda} \bar{L}(W, M_i) \tilde{\nabla}_{\theta} \ln p_{\theta}(M_i).$$

損失を使って重み更新している.

異なるミニバッチの場合の方が,各ネットワークのミニバッチサイズが $1/\lambda$ 倍小さいため,計算時間の点で有利である可能性がある.

しかし,異なるミニバッチは,異なるネットワークの損失を計算するために使用するため,結果の損失関数がレース状態になる可能性がある.この最適化の観点からは,前者の同じミニバッチの場合が好ましい

ベルヌーイ分布によるインスタンス化

柔軟性と効率性を示すためにベルヌーイ分布を考慮し,提案手法が同じフレームワークのもとで最適化できることを目指す

パラメータ W と θ は,学習率 n_W と n_θ で近似した勾配ステップを取ることに
よって更新される.

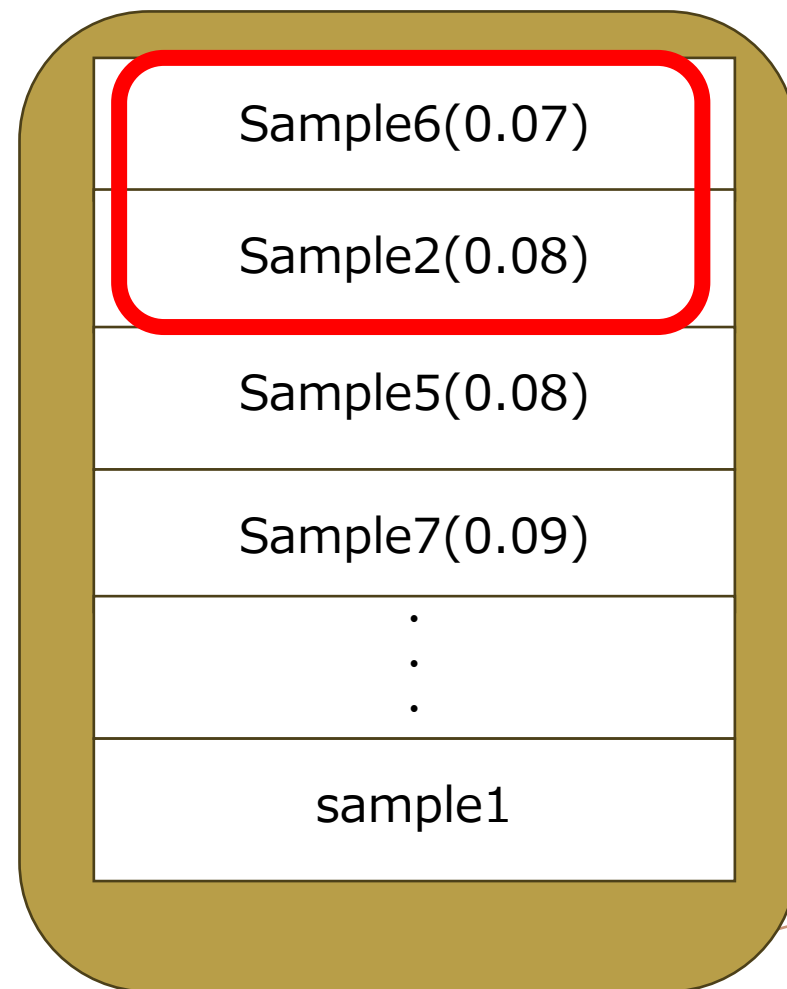
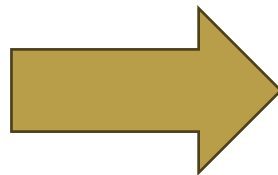
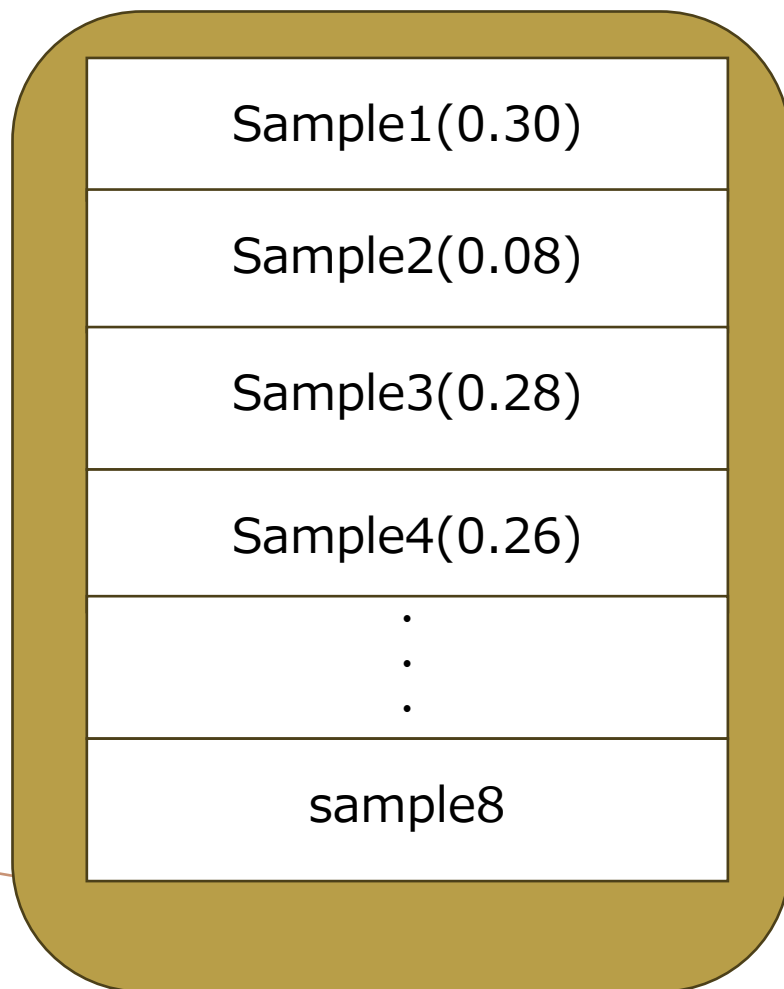
W の更新には,標準的なニューラルネットワークの学習と同様に,確率的勾配
最適化使用することができる.

θ の更新には,各トレーニングステップで損失関数を再スケールするのが簡
単で実用的な方法である.損失値をランキングに基づく効用値 u_i とすると

$$\bar{L}(W, M_i) \rightarrow u_i = \begin{cases} 1 & (best[\lambda/4]samples) \\ -1 & (worst[\lambda/4]samples) \\ 0 & (otherwise) \end{cases}$$

$$\bar{L}(W, M_i) \rightarrow u_i = \begin{cases} 1 & (best[\lambda/4]samples) \\ -1 & (worst[\lambda/4]samples) \\ 0 & (otherwise) \end{cases}$$

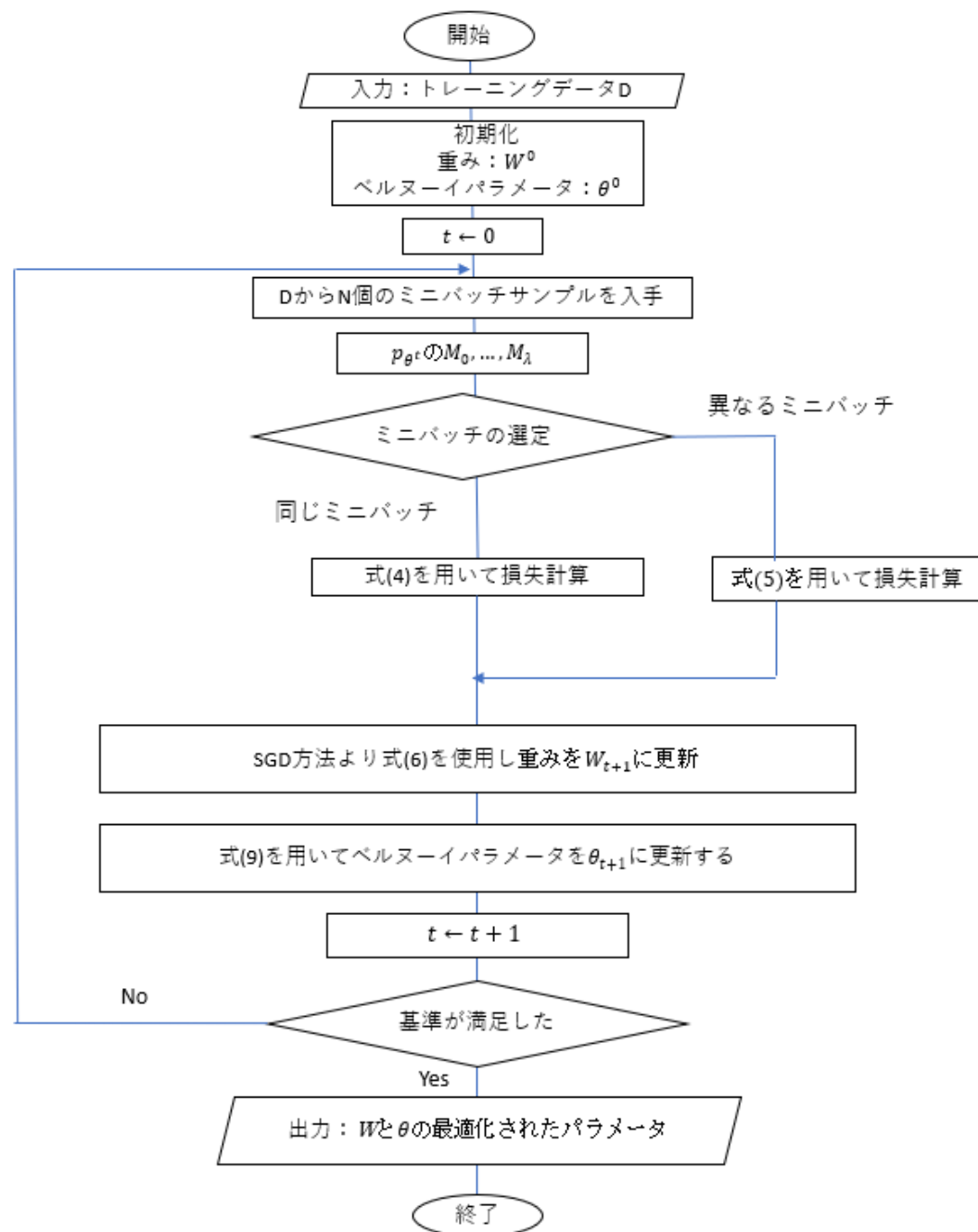
λ : バッチサイズ
 $\lambda = 8$
 best 2 sample



柔軟性と効率性を示すためにベルヌーイ分布を考慮し,提案手法が同じフレームワークのもとで最適化できることを目指す

1. 初期化
2. t の初期化
3. ミニバッチの取得
4. モデルのサンプリング
5. 損失の計算
6. 重みの更新
7. ベルヌーイパラメータの更新
8. t の更新
9. 収束の確認

$$\theta^{t+1} = \theta^t + \frac{n_\theta}{\lambda} \sum_{i=1}^{\lambda} u_i (M_i - \theta^t)$$



実験結果

実験は以下の4つを行った

- 層の選択
- 活性化関数の選択
- 確率ネットワークの適応
- DenseNetsの接続の選択

実験結果

層の選択

目的

同じミニバッチと異なるミニバッチの損失近似の種類の違いを調べ、提案手法が適切な層の数を見つけることができるか確認する

層の選択

実験環境

基盤ネットワークは各層に128個のユニットを持つ完全連結隠れ層とReLUの導関数から構成される.

28 * 28グレースケール画像の60,000学習例を含むMNIST手書き数字データセットを使用する.

また入力層と出力層は,それぞれ784の入力画素値とクラスラベル(0から9)に対応する.

データサンプルのサイズとエポック数は、

- AdaptiveLayer (a) では $N = 64$ と 100
- AdaptiveLayer(b)では $N = 128$ と 200

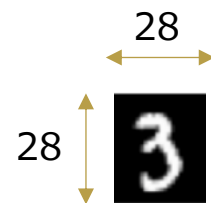
Mの次元は $d = 31$ となる。

層の選択

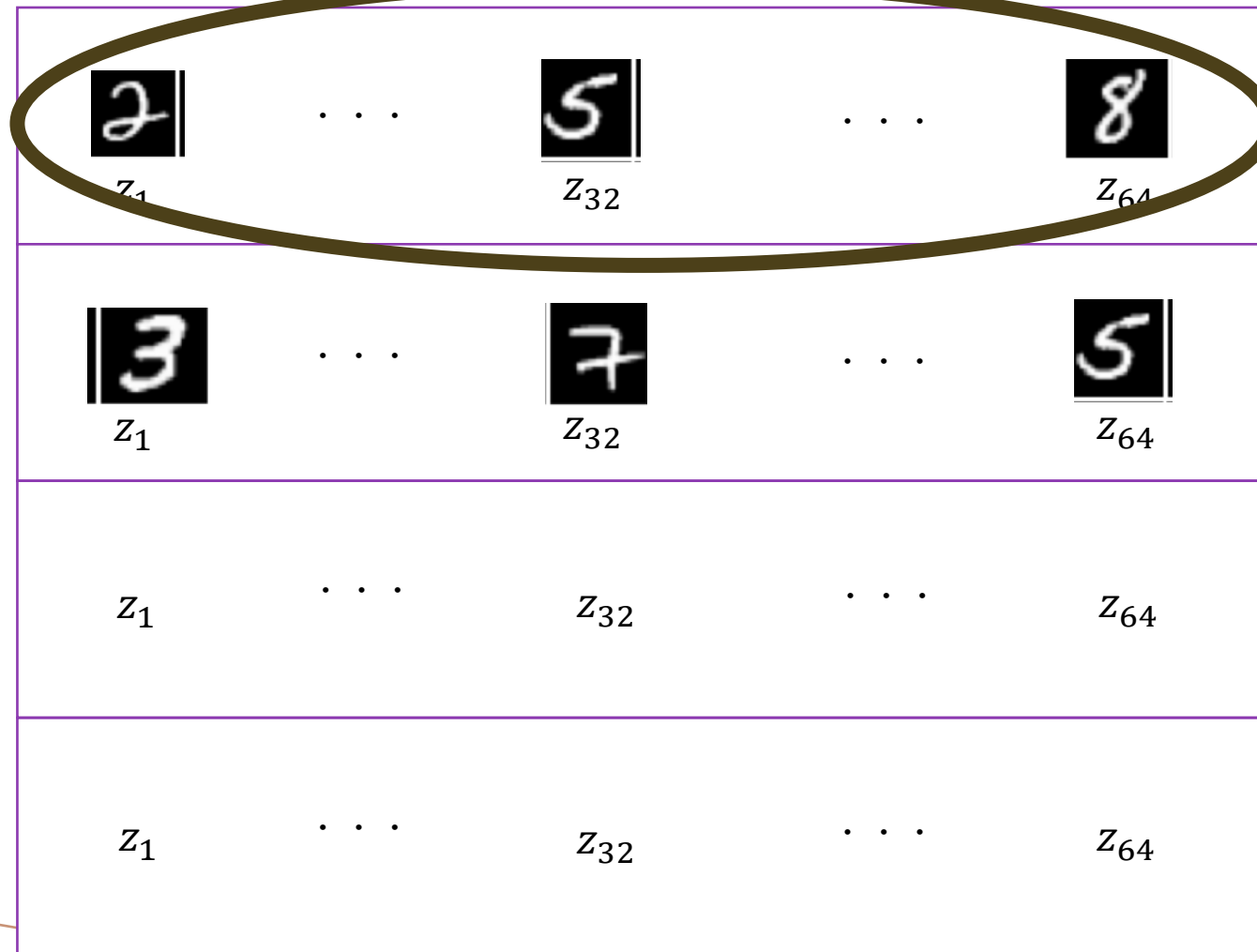
MNIST(60000の訓練例と
10000のテスト例)



⋮



ミニバッチ



層の選択

m は何番目の層かを表している

$$M = (m_1, m_2, \dots, m_d)$$

隠れ層 m_2 より $m_l = 0$ の時

m_{l+1} の層を m_{l-1} に再接続(層の処理をスキップする)

隠れ層 m_2 より $m_l = 1$ の時

接続しない(そのまま処理する)

ReLUの導関数

$$f'(x) = \begin{cases} 0, & x \leq 0 \\ 1, & x > 0 \end{cases}$$

※最初の隠れ層は選択対象にしない→前の隠れ層がないため

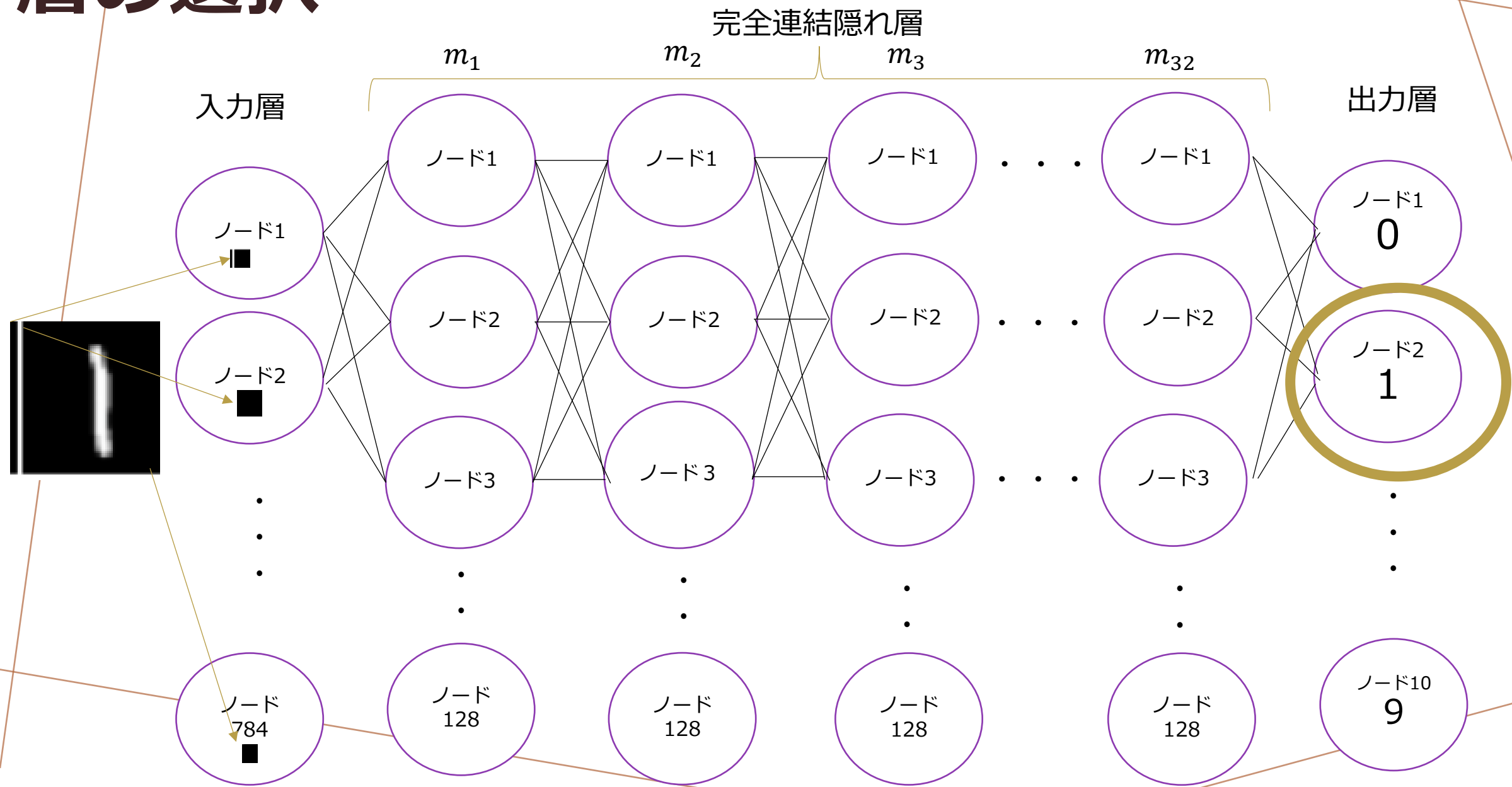
この処理を30回試行する

層の選択

固定された層サイズを持つニューラルネットワーク構造では層の数が21を超えると学習が適切に機能しないため

学習中に隠れ層を22層より少なくする必要がある。→そうしないとちゃんと学習できない→あえて隠れ層を32層用意する

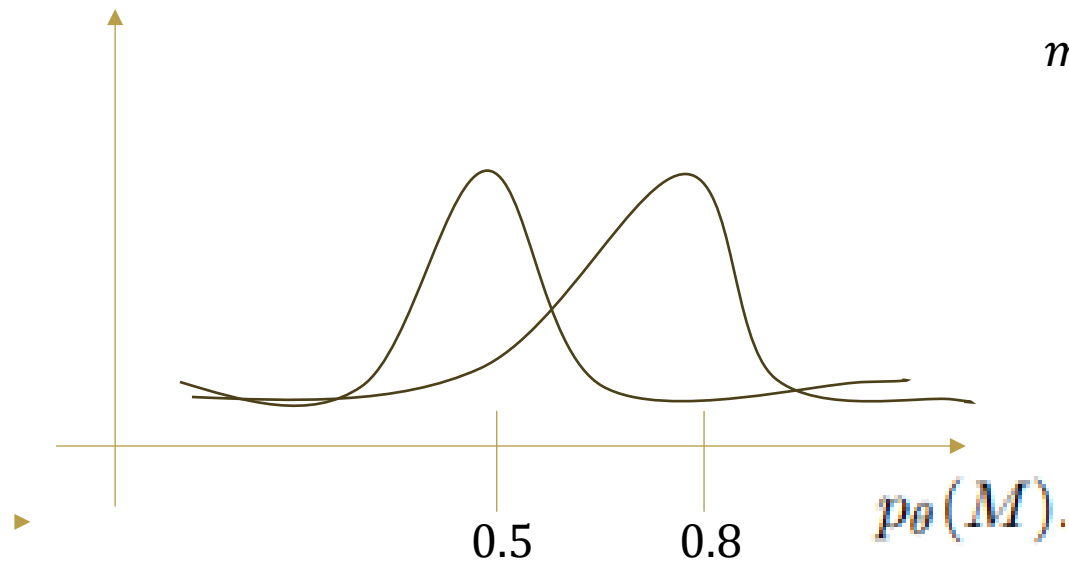
層の選択



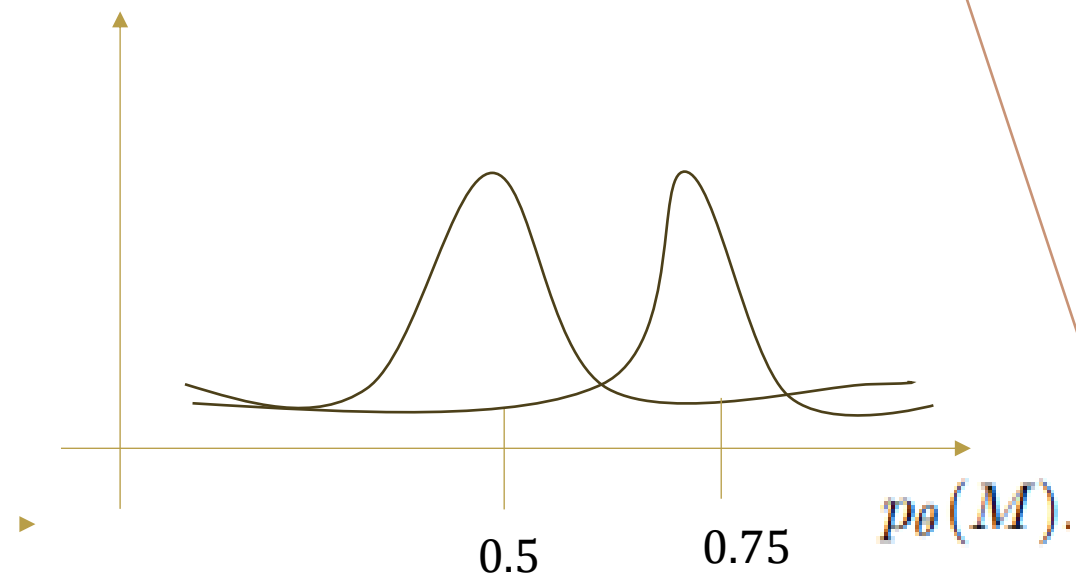
$\theta_{init} = 0.5$ の場合

各隠れ層のパラメータ更新が行われる

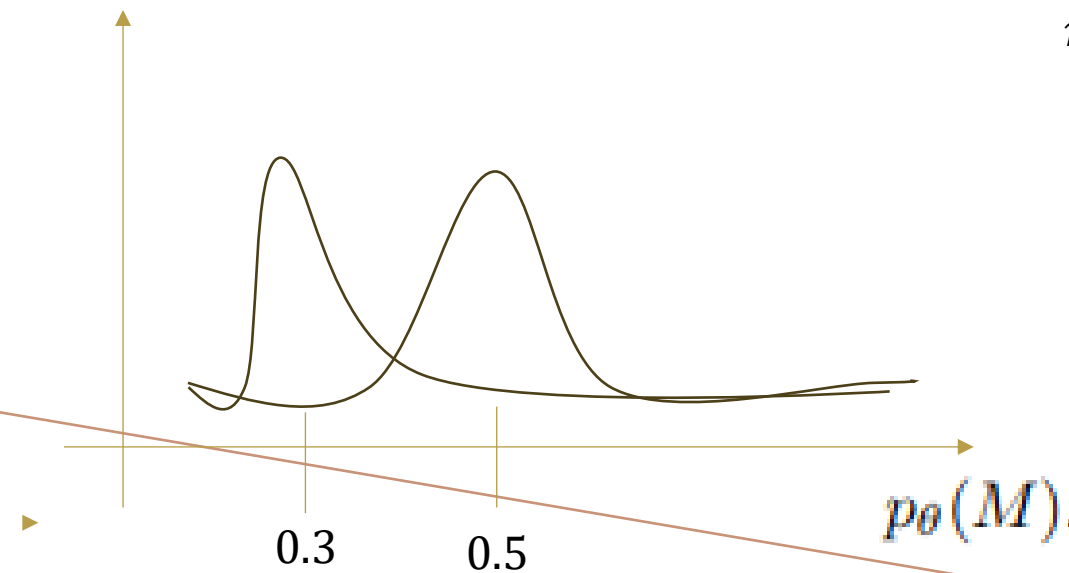
m1



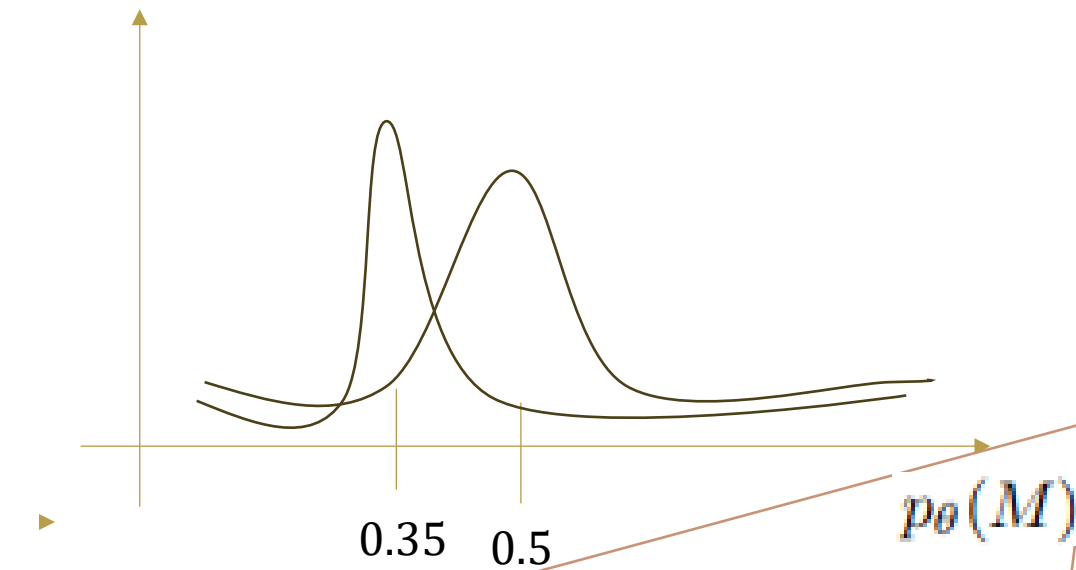
m3



m2



m4



層の選択

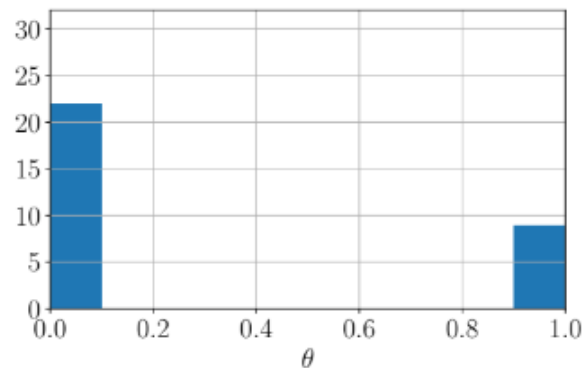


Figure 1: An example of histogram of θ obtained by AdaptiveLayer (a) at the final iteration.

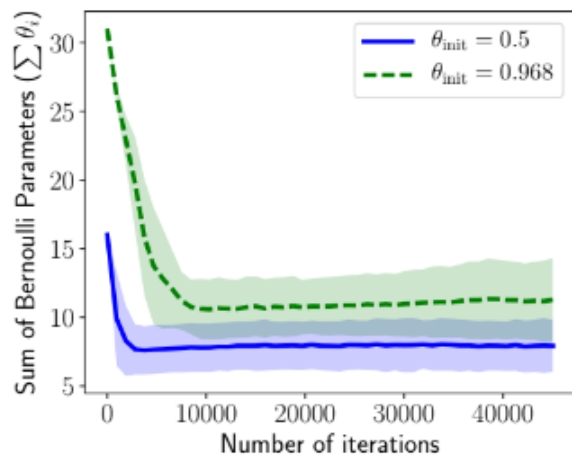


Figure 2: Transitions of the sum of the Bernoulli parameters ($\sum \theta_i$) using AdaptiveLayer (a) when we initialize by $\theta_{\text{init}} = 0.5$ and 0.968 . The expected number of hidden layers is given by $\sum \theta_i + 1$. The first 45,000 iterations (about half of the maximum number of iterations) are plotted.

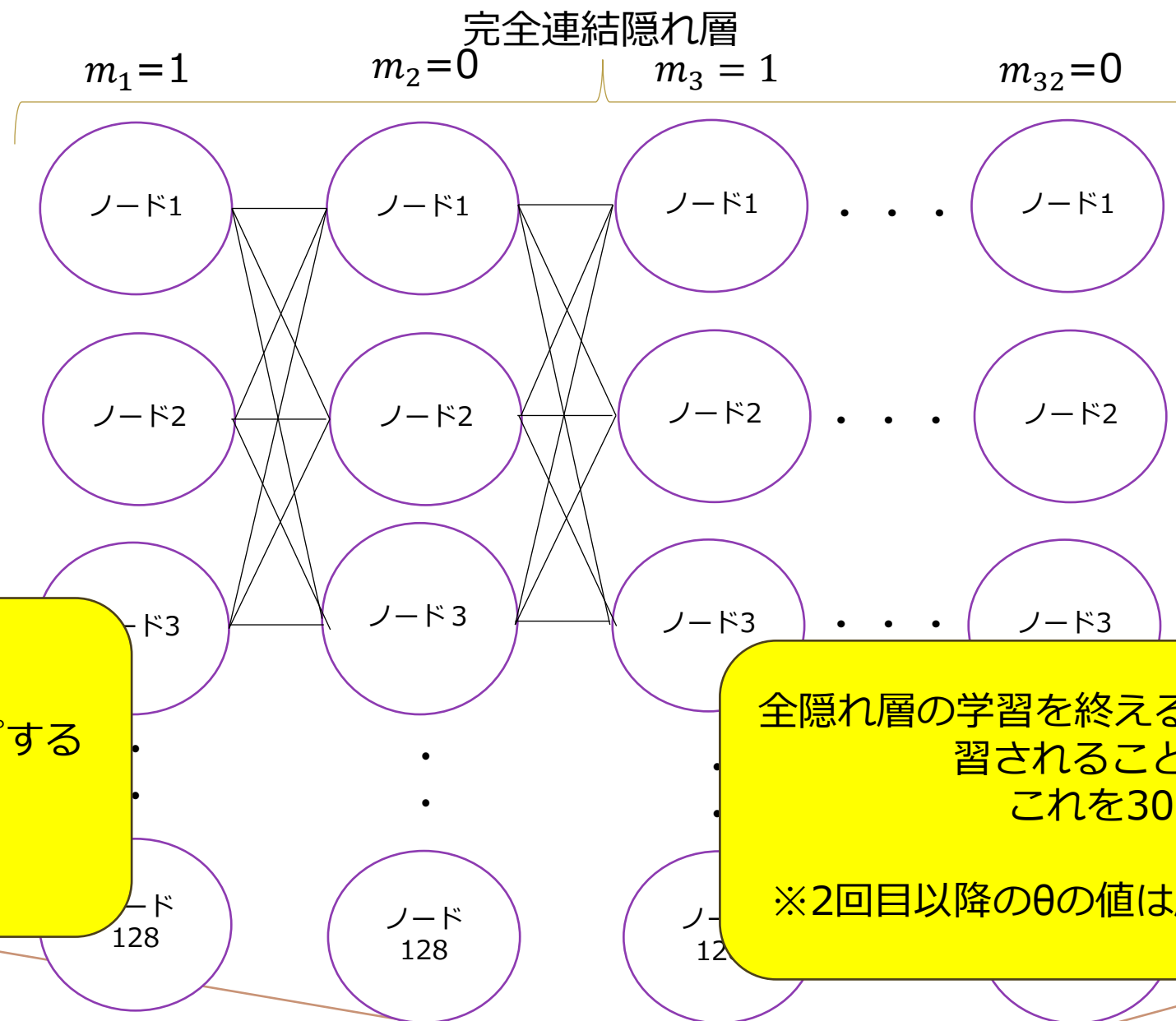
Table 1: Mean test errors (%) over 30 trials at the final iteration in the experiment of selection of layers. The values in parentheses denote the standard deviation.

		$\theta_{\text{init}} = 0.5$		$\theta_{\text{init}} = 0.969$	
		Deterministic	Stochastic	Deterministic	Stochastic
AdaptiveLayer (a)	$(\lambda = 2)$	2.200 (0.125)	2.218 (0.135)	2.366 (0.901)	2.375 (0.889)
AdaptiveLayer (b)	$(\lambda = 2)$	15.69 (29.9)	15.67 (29.9)	35.26 (41.6)	35.27 (41.7)
	$(\lambda = 8)$	2.406 (0.189)	2.423 (0.189)	65.74 (38.8)	65.74 (38.8)
	$(\lambda = 32)$	2.439 (0.224)	2.453 (0.228)	80.59 (24.6)	80.60 (24.6)
	$(\lambda = 128)$	2.394 (0.163)	2.405 (0.173)	80.58 (24.8)	80.58 (24.8)
StochasticLayer		4.704 (0.752)			

$\lambda=2$ を用いた(4)の手法が,初期条件が悪い場合でも最高の性能を示した.

評価 : Adaptivelayar(a)の方がよい

初期値が $\theta = 0.5$ の時



$m_l = 0$
層の処理をスキップする
 $m_l = 1$
接続しない

全隠れ層の学習を終えると1回目は16層が学習されることが分かる
これを30回行う

※2回目以降の θ の値は層ごとに更新される

層の選択

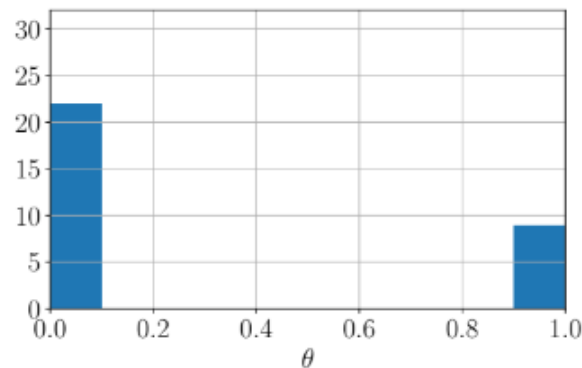


Figure 1: An example of histogram of θ obtained by AdaptiveLayer (a) at the final iteration.

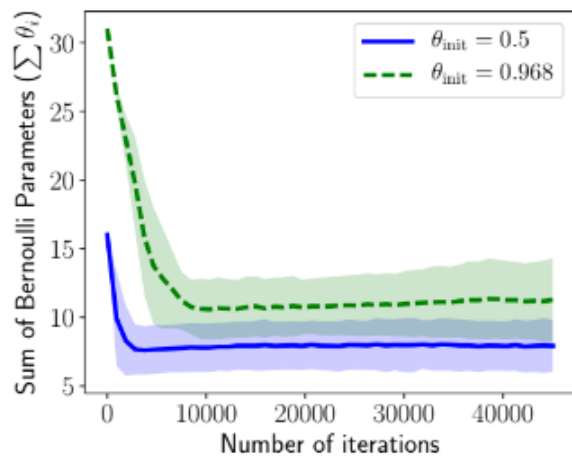


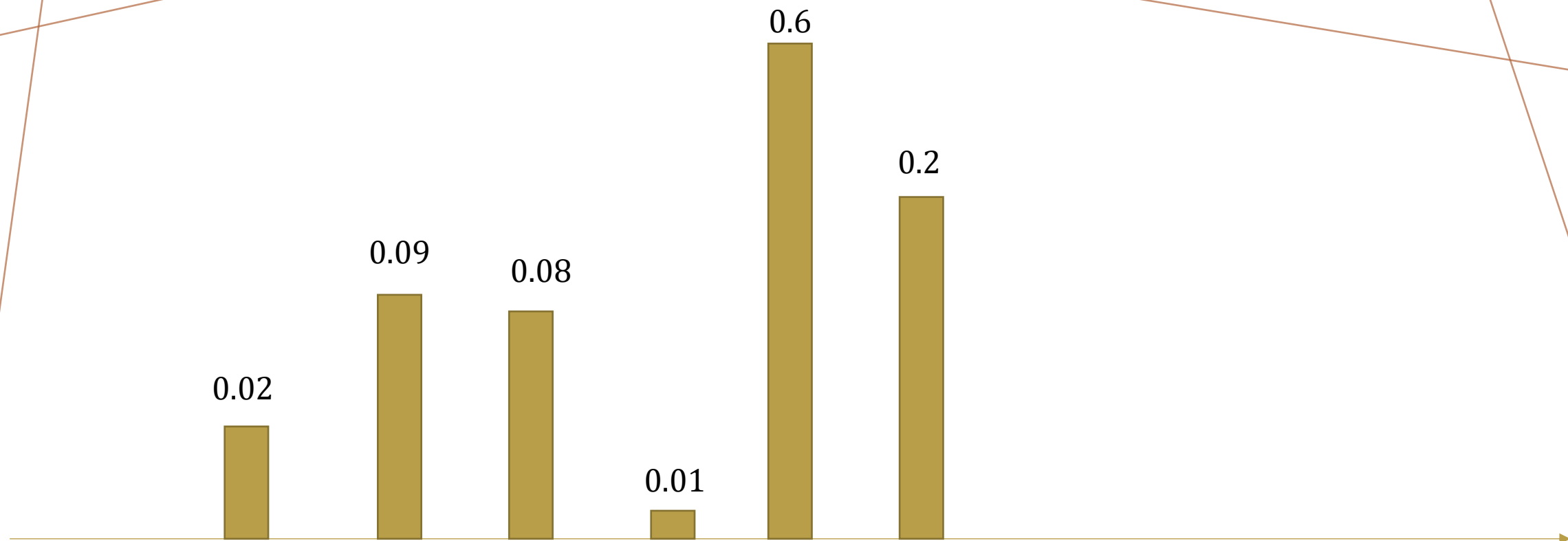
Figure 2: Transitions of the sum of the Bernoulli parameters ($\sum \theta_i$) using AdaptiveLayer (a) when we initialize by $\theta_{\text{init}} = 0.5$ and 0.968 . The expected number of hidden layers is given by $\sum \theta_i + 1$. The first 45,000 iterations (about half of the maximum number of iterations) are plotted.

Table 1: Mean test errors (%) over 30 trials at the final iteration in the experiment of selection of layers. The values in parentheses denote the standard deviation.

		$\theta_{\text{init}} = 0.5$		$\theta_{\text{init}} = 0.969$	
		Deterministic	Stochastic	Deterministic	Stochastic
AdaptiveLayer (a)	$(\lambda = 2)$	2.200 (0.125)	2.218 (0.135)	2.366 (0.901)	2.375 (0.889)
AdaptiveLayer (b)	$(\lambda = 2)$	15.69 (29.9)	15.67 (29.9)	35.26 (41.6)	35.27 (41.7)
	$(\lambda = 8)$	2.406 (0.189)	2.423 (0.189)	65.74 (38.8)	65.74 (38.8)
	$(\lambda = 32)$	2.439 (0.224)	2.453 (0.228)	80.59 (24.6)	80.60 (24.6)
	$(\lambda = 128)$	2.394 (0.163)	2.405 (0.173)	80.58 (24.8)	80.58 (24.8)
StochasticLayer		4.704 (0.752)			

$\lambda=2$ を用いた(4)の手法が,初期条件が悪い場合でも最高の性能を示した.

評価 : Adaptivelayar(a)の方がよい



Deterministic
決定論的

$\theta < 0.5$

$m = 0$

- ドロップアウト
- 層のスキップ
- Tanh
- DenseBlockを接続

$0.5 < \theta$

$m = 1$

- ドロップアウトしない
- 層のスキップしない
- ReLU
- DenseBlockを接続しない

実験結果

活性化関数の選択

目的

活性化関数の選択をして、ネットワーク構造を変化させる

活性化関数の選択

- ・ 実験環境

バイナリベクトル M

ReLU関数 : $m_i = 1$

$M = (m_1, m_2, \dots, m_i)$

tanh関数 : $m_i = 0$

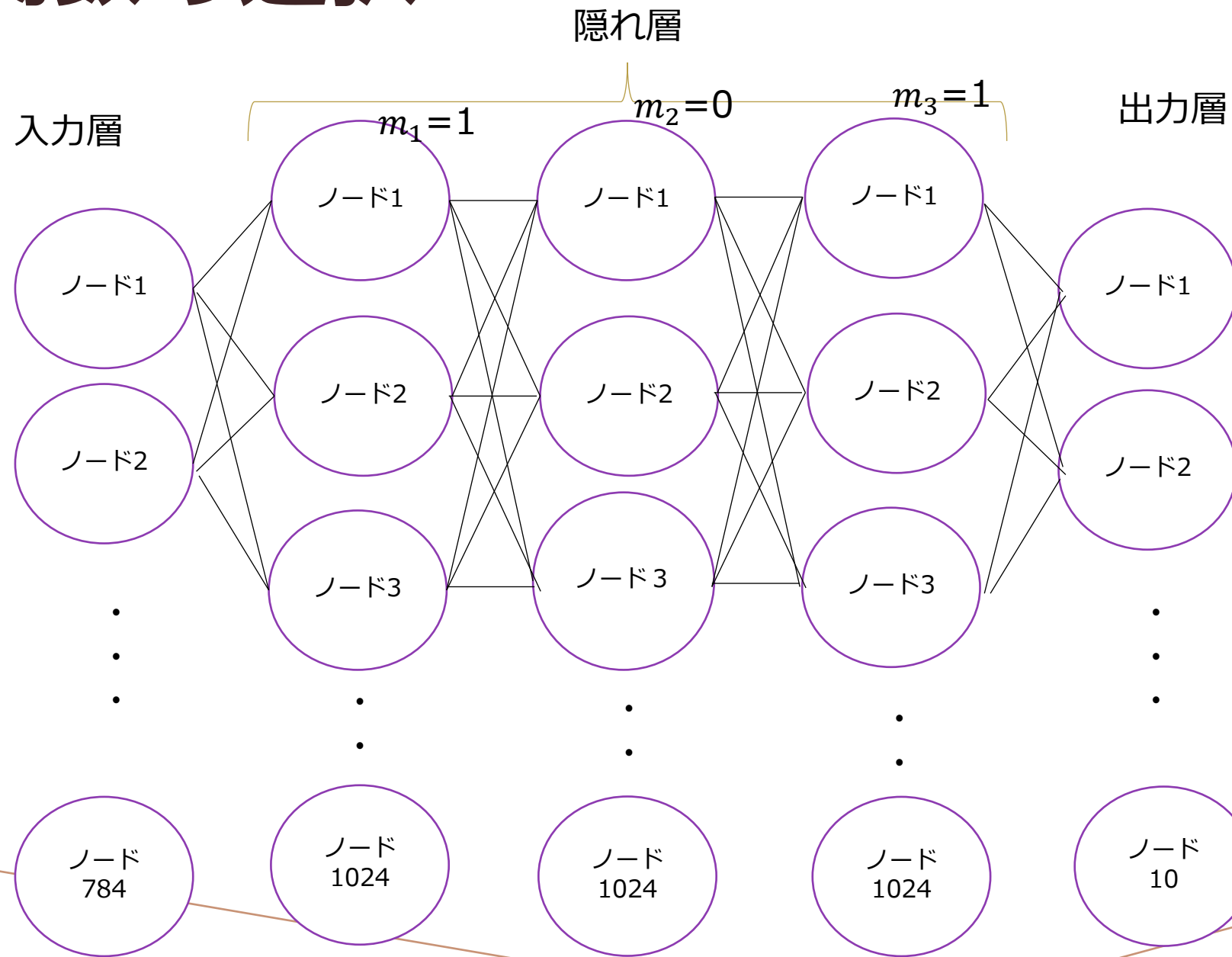
基本ネットワーク構造は、各層に1024ユニットを備えた3つの完全に接続された隠れ層で構成されている

ReLU関数 : 入力値が0以下の場合は、出力値が常に0,

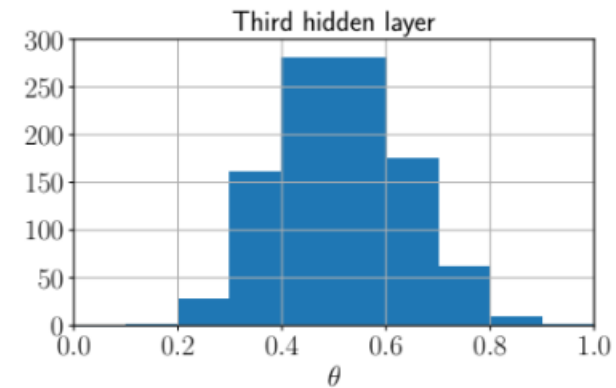
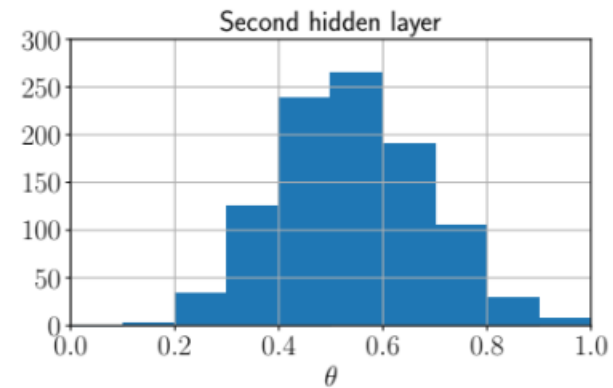
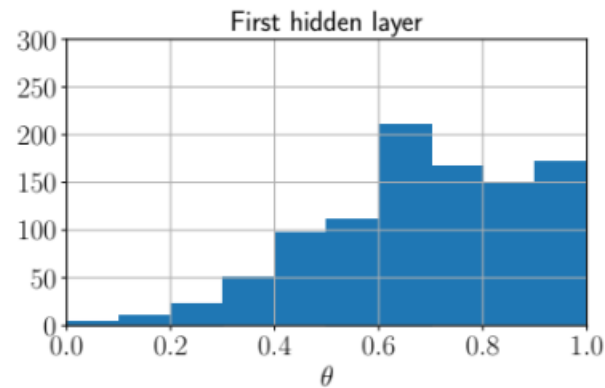
入力値が1以上の場合は、出力値は、入力値と同じ値になる

tanh関数 : あらゆる入力値を-1.0~1.0の範囲の数値に変換して出力する関数

活性化関数の選択



活性化関数の選択



隠れ層の3つの各ノード1024の θ_i の値を表したものの

活性化関数の選択

Table 2: Mean test errors (%) over 30 trials at the final iteration in the experiment of selection of activation functions. The values in parentheses denote the standard deviation. The training time of a typical single run is reported.

	Test error (Deterministic)	Test error (Stochastic)	Training time (min.)
AdaptiveActivation	1.414 (0.054)	1.407 (0.036)	255
StochasticActivation	–	1.452 (0.025)	204
ReLU	1.609 (0.044)	–	120
tanh	1.592 (0.069)	–	120

Adaptive ActivationはStochastic Activationよりもテスト誤差が小さい
提案手法は固定構造ニューラルネットワークと比較して、学習に約2倍の計算時間を必要
とすることが分かる。

活性化関数の選択

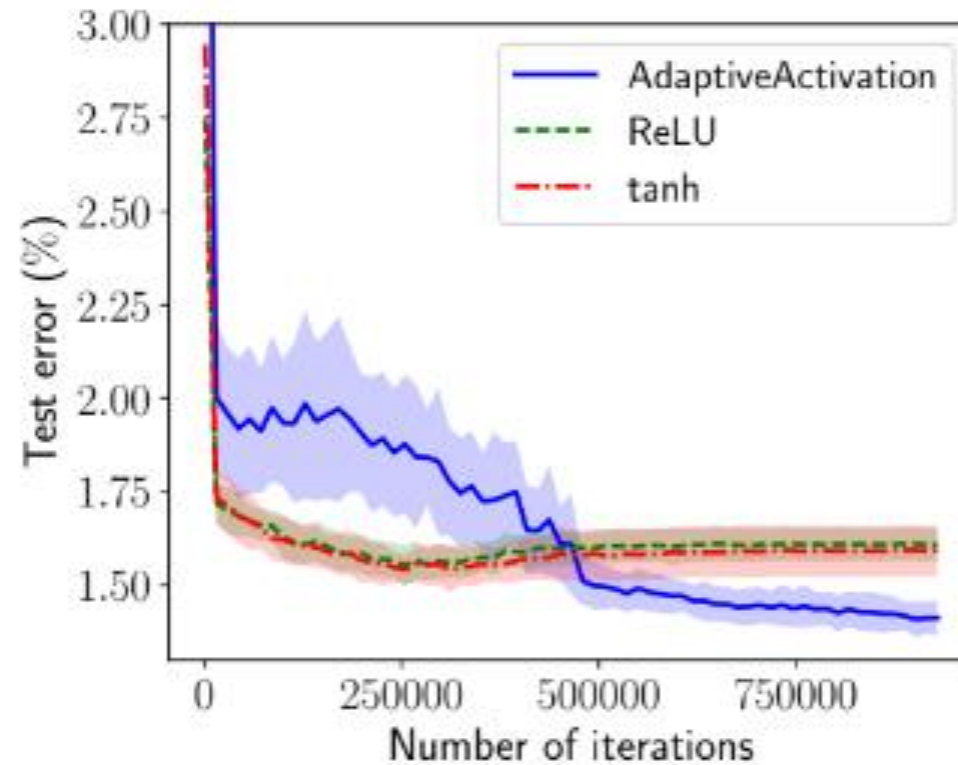


Figure 3: Transitions of the test errors (%) of AdaptiveActivation (deterministic), ReLU, and tanh.

提案手法では反復回数を約45000回すると二つの関数に比べ誤差が低くなる

評価：計算効率を早くすることが目的のため反復回数が半分以下の二つの関数のほうが良いと考える

実験結果

確率ネットワークの適応

目的

ドロップアウト率と層スキップ率を同時に最適化する

過学習を軽減し,ネットワークを広い範囲に共通して適応できるようにする(ドロップアウト)

同時に勾配消失問題を緩和し,勾配がスムーズに伝搬するのを助け,効率的な学習を促進する(レイヤースキップ)

実験結果

確率ネットワークの適応

実験環境

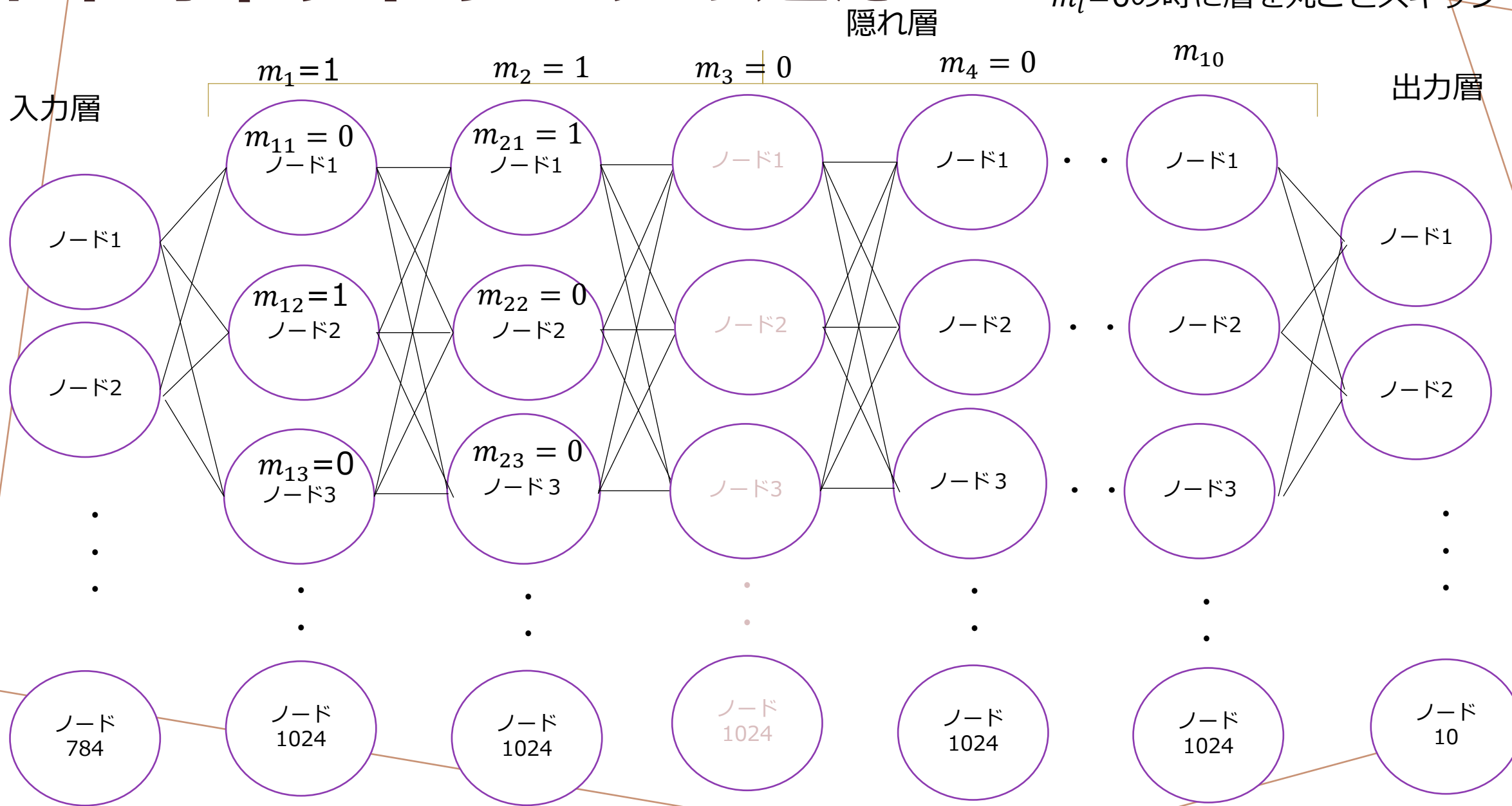
基本ネットワーク構造は,各層に1024ユニットを備えた3つの完全に接続された隠れ層で構成されている

活性化関数の数は $d=3072 \rightarrow 1024*3$,あり

$m_{li} = 0$ の時にドロップアウト, $m_l=0$ の時に層をスキップ

確率的ネットワークの適応

$m_{li} = 0$ の時にドロップアウト,
 $m_l = 0$ の時に層を丸ごとスキップ



確率ネットワークの適応

	Test error (%)	Time (hour)
AdaptiveNet	1.645 (0.072)	1.01
BO (budget=10)	1.780	9.59
BO (budget=20)	1.490	18.29

ベイズ最適化に比べ,提案手法は計算時間が圧倒的に短く誤差率も低い

評価：提案手法の方が圧倒的に良い

実験結果

DenseNetの接続の選択

実験環境

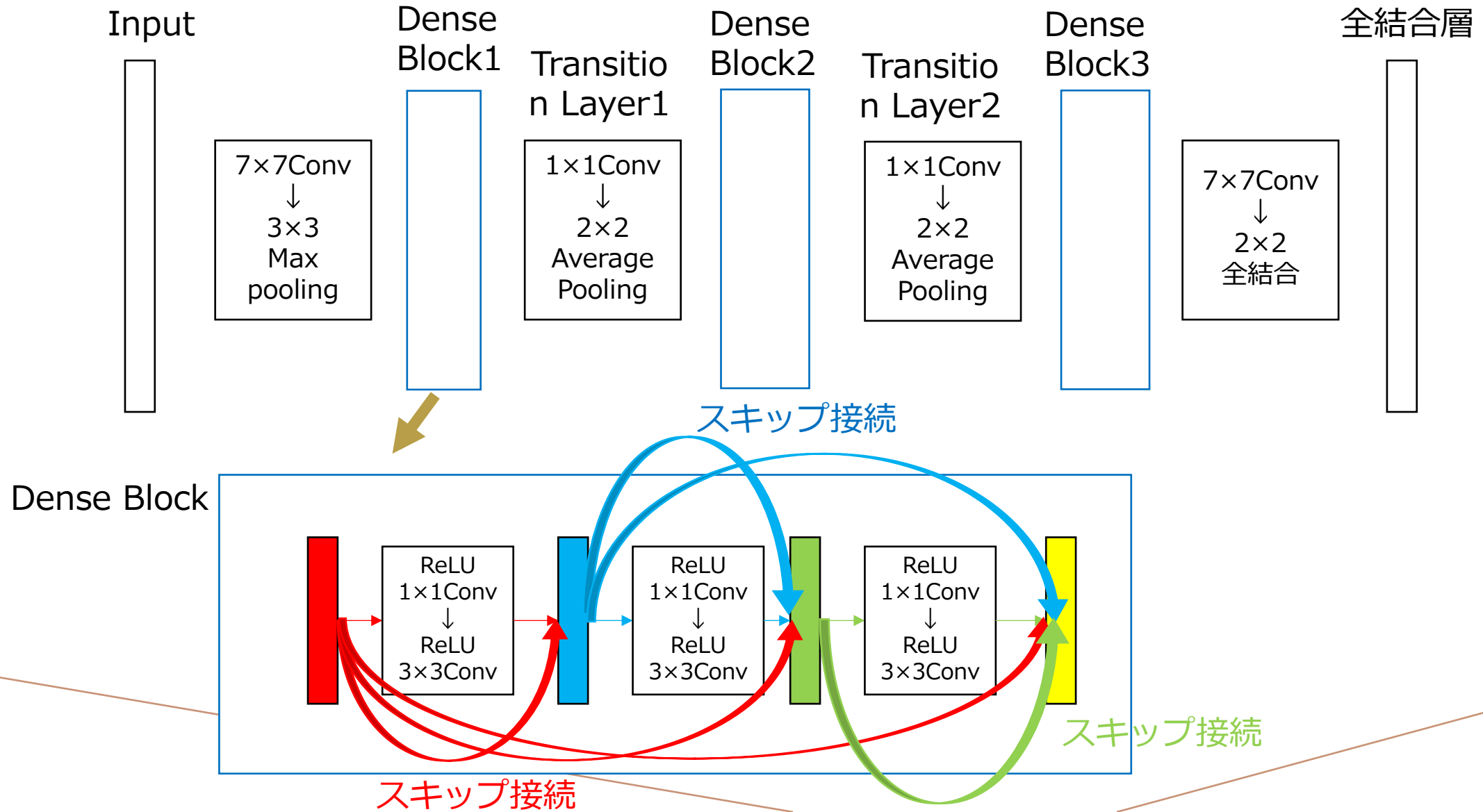
この実験ではCIFAR-10とCIFAR-100を使用する.クラス数が10と100のデータセットであり,トレーニング画像50,000とテスト画像10,000及び画像のサイズは 32×32

DenseNetでの動的最適化→DenseBlockとDenseBlockの層の間の接続をするかしないかを決めている

提案手法→抽出サンプル数 $N=32$,エポック数300

通常デンスネット→抽出サンプル数 $N=64$,エポック数600

実験 4 : DenseNetsにおける接続の選択

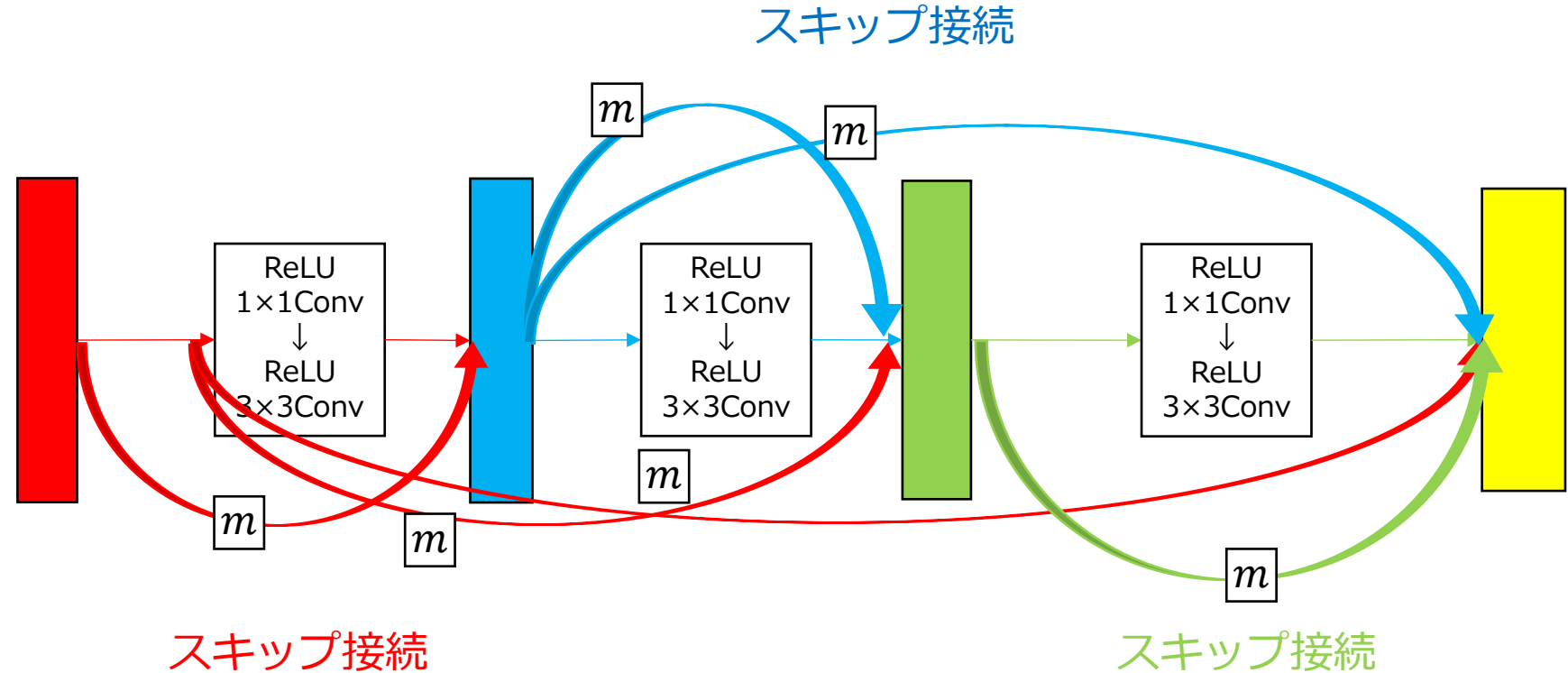
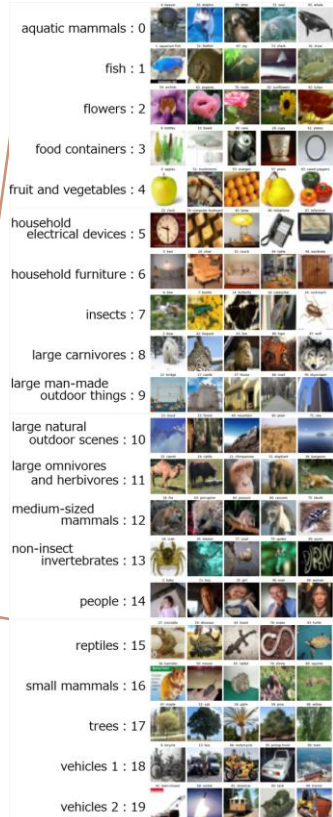


実験 4 : DenseNetsにおける接続の選択

CIFAR - 10



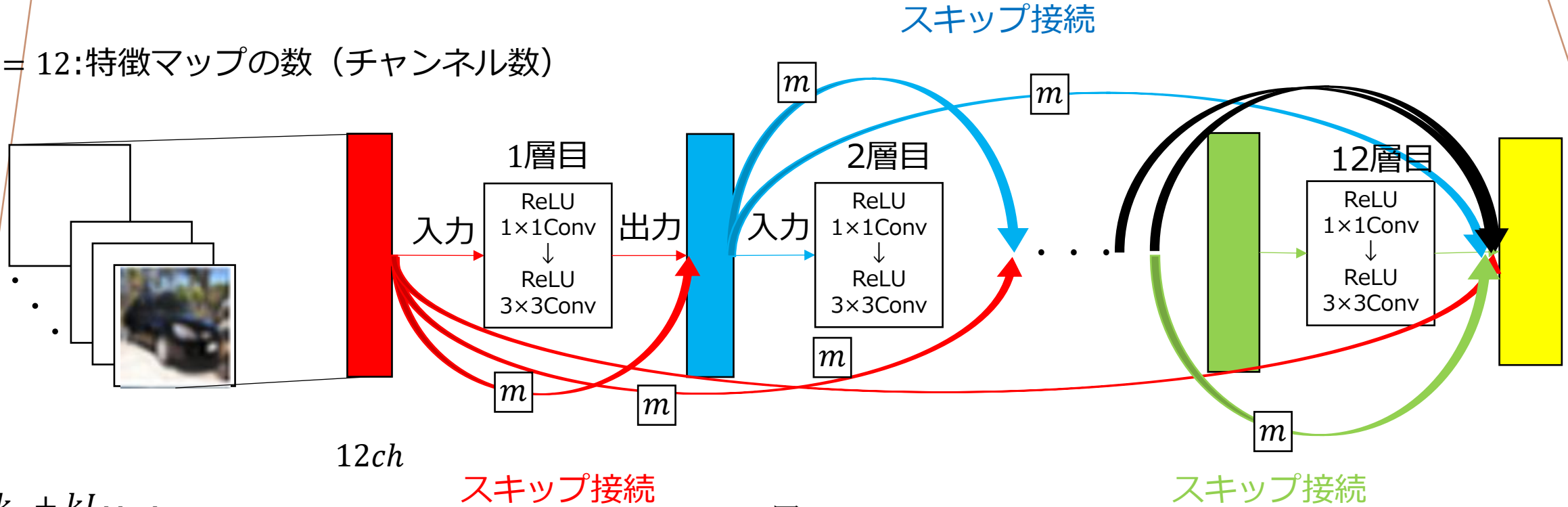
CIFAR - 100



$m = 0$: スキップ接続の削除
 $m = 1$: スキップ接続

実験 4 : DenseNetsにおける接続の選択

$k = 12$: 特徴マップの数 (チャンネル数)



$12ch$

スキップ接続

スキップ接続

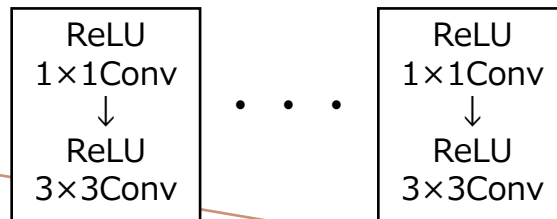
$k_0 + kL_{block}$

k_0 : 入力チャンネル数

k : 出力チャンネル数

L_{block} : DenseBlock内の総数 $L_{block} = 12$

12層



4. DenseNetsの接続の選択

結果のまとめ

CIFAR10 /Deterministic/**提案手法** or **通常Dense Net** → **通常Dense Net**の方がテスト誤差が低い

CIFAR100/Deterministic/**提案手法** or **通常Dense Net** → **提案手法**の方がテスト誤差が低い

CIFAR10 /**Deterministic or Stochastic**/**提案手法** → **Stochastic**の方がテスト誤差が低い

CIFAR100/**Deterministic or Stochastic**/**提案手法** → **Stochastic**の方がテスト誤差が低い

CIFAR10 /"**Deterministic and 通常Dense Net**" or "**Stochastic and 提案手法**"

→ "**Stochastic and 提案手法**"の方がテスト誤差が低い

CIFAR100/"**Deterministic and 通常Dense Net**" or "**Stochastic and 提案手法**"

→ **Stochastic and 提案手法**の方がテスト誤差が低い

→より最適なネットワーク構造を見つけることと
パラメータを最適化することができた。

4. DenseNetsの接続の選択

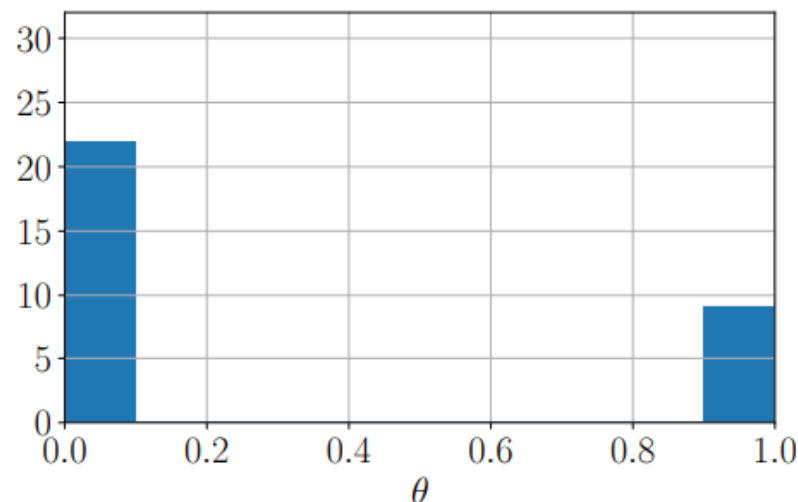


Figure 1: An example of histogram of θ obtained by AdaptiveLayer (a) at the final iteration.

Deterministic

$0.5 < \theta$ だったら...

$m = 0$

→ スキップ接続の削除

$\theta < 0.5$ だったら...

$m = 1$

→ スキップ接続の実行

Stochastic

$\theta = [0.0, 0.2]$

θ が 0.2 だとしたら

$1 - 0.2 = 0.8$

80%の確率で $m = 0$

スキップ接続の**削除**

20%の確率で $m = 1$

スキップ接続の**実行**

$\theta = [0.8, 1.0]$

θ が 0.8 だとしたら

$1 - 0.8 = 0.2$

80%の確率で $m = 1$

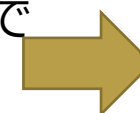
スキップ接続の**実行**

20%の確率で $m = 0$

スキップ接続の**削除**



CIFAR10, 100の両データセットで
高い確率でスキップ接続の削除
されている



さらに、重みパラメータも
10%削減できた

妥当性・改善案

妥当性…4つの実験を行ったが、「活性化関数の選択」については計算効率を考慮するため目的が達成していないと考える.しかし,他の実験では計算効率を早くするといった目的が達成できているため妥当性があると思う

改善案…活性化関数をReLU,tanh関数だけでなく他の関数を加え、比較し,さらに時間短縮を目指す、または、新たにフレームワークを見つけ計算効率を改善する